

Controlled AI-Assisted Development in Student Software Projects: Workflow Control, Failure Modes, and Risk Mitigation

HANAN OSSAMA

Harbin Institute of Technology

School/Department of Artificial Intelligence

Email: 2007ossamahanan@gmail.com

Abstract

AI coding assistants are increasingly used by student developers to generate code, debug errors, refactor software, and accelerate project development. However, in larger student software projects, AI-generated changes may introduce risks that are not always detected by compilation or basic testing. These risks include hallucinated code, incorrect project assumptions, unsafe patches, dependency errors, product drift, hidden logic errors, behavior compatibility issues, usability problems, and scope failure.

This paper presents an empirical case study of AI-assisted development in NeuroNova, a student-built AI learning platform developed with Python and Streamlit. The study compares controlled AI-assisted development with uncontrolled AI-assisted coding across 16 recorded experiments: 10 controlled experiments and 6 uncontrolled experiments. Controlled workflows used read-only inspection, small task decomposition, minimal patches, testing, review, and narrow fixes. Uncontrolled workflows used broader prompts that gave the AI assistant more freedom to modify the project.

In this case study, controlled workflows achieved a 90% observed success rate, while uncontrolled workflows achieved a 66.7% observed success rate. Because the dataset is small and based on one project, these results should be interpreted as early case-study evidence rather than broad statistical proof.

The paper contributes a failure-mode taxonomy, an experiment-level case-study analysis, and a practical controlled workflow for reducing risk exposure in AI-assisted student software development.

Keywords: AI-assisted software development; AI coding assistants; student software projects; human-AI collaboration; software engineering education; workflow control; failure modes.

1. Introduction

AI coding assistants have become common tools in modern software development, with studies and industry reports showing that tools such as GitHub Copilot can improve

developer productivity [1], [5]. Students and developers use tools such as ChatGPT, Codex, GitHub Copilot, Cursor, and other AI-assisted programming systems to generate code, debug errors, explain programming concepts, refactor files, and accelerate project development. For student developers, these tools are especially attractive because they provide immediate support and reduce the difficulty of starting or improving software projects.

However, using AI coding assistants in larger software projects is different from using them for small programming exercises. In a small task, an AI assistant may only need to generate one function, explain one error, or suggest a short code snippet. In a larger project, an AI-generated change must fit into an existing architecture with multiple files, dependencies, user interface components, data structures, APIs, and product requirements. If the AI assistant misunderstands the project context, it may generate code that appears correct but does not fit the real codebase.

This problem became clear during the development of NeuroNova, a student-focused AI learning platform built with Python and Streamlit. NeuroNova includes AI tutoring, study planning, flashcards, exam support, progress tracking, provider fallback logic, and a student-facing interface. Because the project contains multiple connected modules, AI-generated changes can affect more than one part of the system.

During development, AI coding assistants were useful for creating features, improving interface design, debugging dependency problems, and refactoring code. However, they also produced or revealed several risks, including missing dependencies, product drift, hidden logic risks, data-preservation issues, behavior compatibility changes, usability problems, and scope failure. Some AI-generated changes passed compilation and smoke tests but still required controlled review to identify hidden problems.

This paper investigates failure modes and mitigation strategies for AI-assisted development in a larger student software project. Using NeuroNova as a case study, the study compares controlled AI-assisted development with uncontrolled AI-assisted coding across 16 recorded experiments. Controlled workflows used inspection, small task decomposition, minimal changes, testing, review, and narrow correction. Uncontrolled workflows used broader prompts that gave the AI assistant more freedom to modify the project.

The main argument of this paper is that AI coding assistants should not be treated as autonomous software engineers. They are most effective when used as controlled coding partners within a workflow that separates inspection, diagnosis, implementation, testing, review, and minimal correction. This approach allows student developers to benefit from AI speed while preserving human control over architecture, product direction, and final engineering decisions.

This paper studies workflow control as a risk-mitigation strategy for AI-assisted software development in student projects. Rather than evaluating AI coding assistants only by productivity or code-generation ability, the study examines how different prompting and review workflows affect risk exposure, hidden-risk detection, and developer control in a larger evolving codebase.

NeuroNova is used as the empirical case-study setting because it contains multiple interacting components, including user interface modules, AI provider logic, study tools, data handling, prompt behavior, configuration files, and testing scripts. This makes it suitable for observing risks that are unlikely to appear in isolated programming exercises, such as behavior drift, data-preservation risk, dependency failures, product misalignment, and scope failure.

Because this study is based on a small single-project dataset, the paper should be read as an exploratory empirical case study. Its goal is to identify practical risk patterns and workflow strategies rather than to make broad statistical claims about all AI coding assistants or all student software projects.

This study is guided by three research questions:

RQ1: What failure modes appear when AI coding assistants are used in a larger student software project?

RQ2: How do controlled and uncontrolled AI-assisted workflows differ in success rate, failure exposure, and hidden-risk detection?

RQ3: What practical workflow can help student developers use AI coding assistants more safely while preserving human control over software decisions?

1.1 Contributions

The paper makes four contributions. First, it reports an empirical case study of AI-assisted development in NeuroNova, a student-built AI learning platform developed with Python and Streamlit. Second, it presents a failure-mode taxonomy describing risks observed when AI coding assistants are used in a larger student software project, including hallucinated code, context errors, dependency problems, product drift, data-preservation risks, behavior compatibility risks, usability ambiguity, unsafe broad patches, testing assumptions, and scope failure. Third, it compares controlled and uncontrolled AI-assisted workflows across 16 recorded experiments, showing how workflow design affected observed success rate, risk exposure, hidden-risk detection, and minimal targeted fixes in this case-study context. Fourth, it proposes a practical controlled workflow for student developers: Inspect → Diagnose → Implement minimally → Test → Review → Fix narrowly.

1.2 Scope of the Study

This paper does not evaluate all AI coding assistants, all student projects, or all forms of AI-assisted software engineering. Instead, it reports an exploratory empirical case study of workflow control in one larger student software project. The goal is to identify practical risk patterns and workflow mechanisms that can be tested in future multi-project studies.

2. Related Work

Research on AI coding assistants has expanded as tools such as ChatGPT, Codex, GitHub Copilot, Cursor, and other large language model-based systems have become more common in software development and programming education. Existing work can be grouped into five main areas: developer productivity, code generation, programming education, human-AI collaboration, and limitations of AI-generated code.

ChatGPT is one example of a conversational AI system used for interactive assistance, explanation, and code-related support [4].

2.1 AI Coding Assistants and Developer Productivity

One major area of research focuses on whether AI coding assistants improve developer productivity [1], [5], [6]. AI coding tools can help developers generate code faster, complete repetitive programming tasks, and reduce the time needed to start implementation. For students, this can be especially useful because AI tools provide immediate support when they face unfamiliar syntax, errors, or project structures.

However, productivity alone does not measure software quality or safety. A task may be completed faster, but the generated code may still require review, testing, correction, or later maintenance. This paper therefore focuses not only on whether AI tools help development move faster, but also on whether their outputs are safe in larger student software projects.

2.2 Large Language Models for Code Generation

Large language models have been widely used for code generation, debugging, program repair, documentation, and explanation. These models can generate code from natural language instructions and can help developers understand errors or unfamiliar programming patterns.

However, generated code may contain incorrect assumptions, incomplete logic, missing dependencies, hallucinated functions, or mismatches with the existing project architecture. These problems become more serious in larger projects because the AI assistant must understand file relationships, existing functions, dependencies, user interface logic, and long-term maintainability.

2.3 AI Tools in Programming Education

AI tools are increasingly used by students to learn programming [7], [8], [9]. Students may use them to explain concepts, debug errors, generate examples, and receive feedback. This can support learning, especially for beginners who need step-by-step explanations.

At the same time, AI use in programming education creates risks. Students may overtrust AI-generated output, accept code without understanding it, or allow AI tools to make project-level decisions. In larger student projects, this can reduce learning and create

unstable software if the student does not use version control, testing, and review practices.

2.4 Human-AI Collaboration in Software Development

Human-AI collaboration research emphasizes that AI assistants should support developers rather than replace them [10], [11]. This idea is central to the present study. The proposed workflow treats AI coding assistants as controlled coding partners rather than autonomous software engineers. The human developer remains responsible for project direction, architecture, testing, product vision, and final decision-making.

2.5 Limitations of AI Coding Assistants

AI coding assistants can make mistakes that are difficult to detect immediately, including hallucinated code, incorrect imports, missing dependencies, unsafe refactoring, behavior drift, usability problems, and security weaknesses [12], [13]. The NeuroNova case study adds practical evidence to this area by showing that AI-generated changes can pass compilation and smoke tests while still containing hidden risks such as data-preservation issues, behavior compatibility problems, and UI label ambiguity.

2.6 Research Gap

Much existing discussion around AI coding assistants focuses on productivity, code generation ability, or educational support. Less attention is given to practical workflows that help student developers safely use AI coding assistants in larger evolving projects.

This paper addresses that gap by studying failure modes and mitigation strategies in a real student software project. Instead of asking only whether AI makes development faster, this research asks how AI-assisted coding can be controlled to reduce risk.

3. Analytical Framing: Workflow Control and Risk Exposure

This study is based on the idea that AI-assisted development risk is shaped not only by the quality of the AI-generated output, but also by the workflow used to request, inspect, test, and accept that output. In larger software projects, an AI-generated change can be technically correct in isolation but risky in context if it affects multiple files, changes expected behavior, introduces dependency assumptions, or shifts product direction.

The paper therefore treats workflow control as a central analytical concept. A controlled workflow limits the AI assistant's action space through read-only inspection, narrow prompts, minimal edits, explicit testing, and review before acceptance. An uncontrolled workflow gives the AI assistant broader freedom to decide implementation scope, modify multiple files, or propose larger architectural or product-level changes.

Risk exposure is used to describe how much potential project impact an AI-assisted change creates. A change has higher risk exposure when it affects more files, touches core logic, changes user-facing behavior, modifies shared helper functions, or requires the developer to understand a larger patch before accepting it. This framing helps explain why an AI-generated change can pass basic tests while still requiring controlled review.

Using this framing, the study analyzes the NeuroNova experiments not only by success or failure, but also by how workflow design affected reviewability, hidden-risk detection, reversibility, and human control.

4. Case Study: NeuroNova

NeuroNova is a student-built AI learning platform developed mainly with Python and Streamlit. The project is designed to help students study more effectively through AI tutoring, study planning, flashcards, exam support, progress tracking, multilingual learning support, and provider fallback logic.

NeuroNova was selected as the case study because it is larger than a simple programming exercise. It contains multiple connected components, including user interface pages, AI provider logic, study tools, data handling, prompt behavior, configuration files, and testing scripts. Because these components are connected, a change in one part of the project can affect other parts of the system.

The project includes several main components: user interface pages, AI provider and fallback logic, study planner tools, flashcard and review features, exam mode, progress dashboard, prompt and tutor behavior logic, configuration and dependency management, and smoke tests and compilation checks. This makes NeuroNova useful for studying AI-assisted development risks. In a small coding task, an AI assistant may only need to generate a function or explain an error. In NeuroNova, however, an AI-generated change may affect routing, state management, dependencies, user interface behavior, provider selection, or study-tool integration.

During the development of NeuroNova, AI coding assistants were used for different types of tasks, including debugging, feature integration, refactoring, UI redesign, provider fallback improvement, and code review. Some AI-generated changes were useful and passed tests. Other changes produced or revealed risks such as missing dependencies, product drift, behavior compatibility changes, hidden data-preservation risks, usability problems, and scope failure.

The case study is not intended to represent all software projects or all AI coding assistants. Instead, it provides a practical example of how AI-assisted development behaves in one real student project. The goal is to identify patterns that may help other student developers use AI coding assistants more safely in larger projects.

This case-study approach is appropriate because the research investigates AI-assisted development behavior in its real project context rather than in an isolated programming exercise [3].

5. Methodology

This research uses an empirical case study methodology, following the general logic of case study research in software engineering [3]. The case study is NeuroNova, a student-built AI learning platform developed with Python and Streamlit. The purpose of the study is to observe how AI coding assistants behave when used in a larger student software project and to compare uncontrolled AI-assisted coding with controlled AI-assisted development.

5.1 Research Design

The study is based on 16 recorded AI-assisted development experiments. Each experiment involved using an AI coding assistant to perform, review, or help analyze a task related to NeuroNova.

The experiments were divided into two workflow categories:

Table 1. Controlled and uncontrolled workflow definitions.

Workflow Type	Description
Controlled AI-assisted development	Narrow prompts with strict rules, read-only inspection, minimal edits, testing, and review
Uncontrolled AI-assisted coding	Broad prompts that allowed the AI to edit any files needed or make larger changes

The study does not attempt to evaluate every AI coding assistant or every type of software project. Instead, it focuses on practical failure modes and mitigation strategies in one real student project.

5.2 Controlled Workflow

Controlled AI-assisted development was defined by prompts that limited the AI assistant's freedom. Controlled prompts often included constraints such as: do not edit files, inspect first, change only one file, do not refactor, do not rename functions, preserve existing behavior, make the smallest safe fix, provide testing steps, and provide a rollback plan.

Controlled experiments were used for diagnosis, review, minimal bug fixes, compatibility fixes, and UI text corrections.

5.3 Uncontrolled Workflow

Uncontrolled AI-assisted coding was defined by broader prompts that gave the AI assistant more freedom. Uncontrolled prompts included broader requests such as improving the system, refactoring the codebase, redesigning the interface, improving provider fallback logic, fully integrating multiple features, and generating a large multi-phase skeleton.

These tasks were useful for testing how AI behaves when given more autonomy. Some uncontrolled experiments succeeded, but they often touched multiple files or affected important system areas, increasing risk exposure.

5.4 Data Collection

Each experiment was recorded in a structured log. The recorded fields included:

Table 2. Structured fields used in the experiment log.

Field	Description
Experiment ID	Unique experiment identifier
Date	Date of the experiment
AI Tool	AI tool used, such as ChatGPT or Codex
Workflow	Controlled or uncontrolled
Task Type	Debugging, refactoring, UI redesign, provider refactor, review, or fix
Task Summary	Summary of the prompt or task
Files Changed	Number of files changed
Failure Mode	Type of issue observed
Severity	None, Low, Medium, High, or Critical
Result	Successful, failed, rejected, or successful review
Accepted	Whether the output was accepted
Notes	Additional observations

A cleaned dataset was created from the raw experiment log. The clean dataset includes only the 16 final experiments used in the analysis.

5.5 Experiment Dataset Overview

The final dataset contains 16 recorded AI-assisted development experiments. Each experiment was assigned an identifier, workflow type, task category, observed result, failure mode, and severity level. The dataset was cleaned from the raw development log to include only experiments with enough information for analysis.

The purpose of this dataset is not to provide a statistically representative sample of all AI-assisted development tasks. Instead, it provides a structured record of observed AI-assisted development behavior in one real student software project.

A complete experiment table should be included in the appendix or public reproducibility repository. The table should include the following fields:

Table 3. Dataset fields used for the experiment-level summary.

Field	Purpose
Experiment ID	Tracks each recorded experiment
Workflow type	Separates controlled and uncontrolled AI-assisted workflows
Task category	Describes whether the task involved debugging, refactoring, UI redesign, integration, review, or correction
AI tool	Identifies the tool used, such as ChatGPT or Codex
Files changed	Helps estimate patch size and risk exposure
Result	Records whether the experiment was successful, failed, rejected, or review-only
Failure mode	Links the experiment to the failure-mode taxonomy
Severity	Classifies the impact of the observed issue
Accepted	Records whether the output was accepted into the project
Notes	Preserves qualitative observations from the development process

Including this dataset structure improves transparency and helps readers understand how the results were produced.

The final cleaned dataset includes the experiment IDs C6, C7, C8, C9, U3, U4, C10, C11, U5, C12, C13, U6, C14, C15, U7, and U8.

5.6 Severity Classification

Failures were classified by severity:

Table 4. Severity classification used in the study.

Severity	Meaning
None	No failure observed
Low	Minor issue with little impact
Medium	Hidden risk or usability/behavior issue while the app still works
High	Serious issue that may affect major functionality or project direction
Critical	Application cannot start or a core feature is broken

5.7 Evaluation Criteria

The experiments were evaluated using both quantitative and qualitative criteria.

Quantitative criteria included the number of controlled and uncontrolled tasks, success rate, failure or rejection rate, number of high or critical failures, number of hidden risks found during review, and number of minimal targeted fixes.

Qualitative criteria included the type of failure mode, whether the AI respected project context, whether the AI made risky assumptions, whether the change affected architecture, whether the change preserved existing behavior, and whether controlled review improved safety.

5.8 Success, Failure, and Hidden-Risk Definitions

An experiment was classified as successful if the AI-assisted output achieved the intended task, integrated with the existing project context, and did not introduce an observed blocking error during testing or review.

An experiment was classified as failed if the AI-assisted output did not solve the intended task, produced incomplete or unrelated output, broke important functionality, or could not be used in the project.

An experiment was classified as rejected if the output technically worked or appeared useful but was not accepted because it introduced unacceptable project risk, unnecessary complexity, product drift, or behavior that did not match the intended direction of NeuroNova.

A hidden risk was defined as a problem that was not immediately detected by compilation or basic smoke testing but was later identified through controlled review. Examples included data-preservation risk, behavior compatibility drift, usability ambiguity, dependency risk, and unsafe broad changes.

This classification was based on the developer's review of the experiment logs, changed files, test results, and observed project behavior. Because the classification involved human judgment, the results should be interpreted as case-study evidence rather than as fully objective measurement.

5.9 Coding Procedure

Each experiment was reviewed after completion and coded using the structured experiment log. The coding process involved four steps.

First, the experiment was classified by workflow type as either controlled or uncontrolled. Controlled experiments used narrow prompts, read-only inspection, minimal edits, testing, or review. Uncontrolled experiments used broader prompts that allowed the AI assistant to make larger changes or decide implementation scope.

Second, the experiment result was classified as successful, failed, rejected, or successful review. This classification was based on whether the AI-assisted output achieved the intended task, whether it integrated with the existing project, whether tests passed when applicable, and whether the output was accepted into the project.

Third, any observed risk was assigned to one or more failure-mode categories from the taxonomy. For example, missing package problems were coded as dependency errors, unverified invented functions were coded as hallucinated code, broad changes affecting multiple files were coded as unsafe large patches or broad-change risks, and incomplete responses to large multi-part tasks were coded as scope failure.

Fourth, severity was assigned using the severity scale defined in Section 5.6. Issues that did not affect project behavior were classified as low or none. Hidden risks that required review but did not immediately break the app were classified as medium. Issues that affected major functionality, project direction, or startup behavior were classified as high or critical.

Because the coding was performed by the developer-researcher, the classification may contain subjectivity. To reduce this risk, the paper reports the cleaned experiment-level summary in Appendix A and discusses validity threats in Section 10. Future work should involve independent reviewers and inter-rater agreement.

5.10 Coding Rubric

The following rubric was used to classify experiment outcomes and risks.

Coding Category	Decision Rule
Successful	The AI-assisted output achieved the intended task, integrated with the project, and did not introduce an observed blocking problem
Failed	The output did not solve the task, was incomplete, unrelated, or broke important functionality
Rejected	The output appeared useful but was not accepted because it introduced unacceptable risk, complexity, product drift, or behavior mismatch
Hidden risk	The issue was not detected by compilation or smoke testing but was later found through review
Controlled workflow	The prompt limited the AI through inspection, narrow scope, minimal edits, testing, or review
Uncontrolled workflow	The prompt allowed broad edits, multi-file changes, feature-level decisions, or large implementation scope

5.11 Researcher Role and Reflexivity

The study was conducted by the developer-researcher who built and maintained the NeuroNova project. This provided direct access to the project context, development decisions, AI-generated outputs, test results, and observed risks. This position was useful because many risks, such as behavior drift, product misalignment, or data-preservation problems, required knowledge of the project's intended behavior.

However, the developer-researcher role also introduces possible bias. The same person who used the AI tools also classified the experiment outcomes and interpreted the results. To reduce this risk, the study used a structured experiment log, explicit success and failure definitions, a severity scale, and an experiment-level appendix. Future work should involve independent reviewers, inter-rater agreement, and replication across additional projects.

5.12 Methodological Note

This study should be interpreted as an empirical case study rather than a controlled statistical experiment. The dataset is small, based on one student-built project, and classified by the developer-researcher. Therefore, the results are not intended to prove that controlled AI-assisted workflows always outperform uncontrolled workflows.

Instead, the purpose of the study is to identify risk patterns observed during real AI-assisted development and to examine how workflow control affected reviewability, hidden-risk detection, and developer decision-making in this project context. The study’s main value is therefore practical and exploratory: it provides structured evidence from a real student software project and proposes a workflow that can be tested more rigorously in future work.

A detailed discussion of validity threats is provided in Section 10.

5.13 Reproducibility Materials

To support reproducibility, the study can be accompanied by a public research repository containing the cleaned experiment dataset, failure-mode definitions, selected anonymized prompts, experiment summaries, and paper versions. The repository should separate accepted outputs, rejected outputs, and review notes where possible. Because the case study is based on a real student project, any sensitive API keys, private configuration files, or personal information should be removed before public release.

A reproducibility package would allow other student developers or researchers to inspect how experiments were classified, compare controlled and uncontrolled prompts, and apply the proposed workflow to their own projects.

6. Failure-Mode Taxonomy

Based on the NeuroNova experiments, this research classifies AI coding assistant risks into a failure-mode taxonomy. The purpose of this taxonomy is to organize the types of problems observed during AI-assisted development and connect them to concrete project examples. It also reflects broader concerns that AI-generated code can contain weaknesses that are not immediately visible to the developer [12], [13].

Table 5. Failure-mode taxonomy observed in the NeuroNova case study.

Failure Mode	Main Risk	Example from NeuroNova
Hallucinated Code	Invented functions, files, or structures	Suggested <code>render_flashcard_analytics()</code> without verifying the real project
Context Error	Incorrect assumption about project structure	Suggested code locations that did not match the active file structure
Integration Error	Problem between connected modules	Gamification integration revealed a dependency/import problem
Dependency Error	Missing or misconfigured package/import	<code>wikipedia_api.py</code> depended on the missing <code>wikipedia</code> package
Product Drift	Shift away from the requested task	Asked for spaced repetition, but AI suggested Flashcard Analytics
Decision-Making Limitation	Weak project-level judgment	Suggested advanced upgrades before verifying project priorities
Data Preservation Risk	Possible loss of user-created data	<code>normalized[:7]</code> could remove manual/completed planner tasks
Behavior Compatibility Risk	Refactor changes expected behavior	Exam Mode fallback behavior changed from <code>file_type</code> to <code>title first</code>
UI Label Ambiguity	Confusing navigation or labels	Home was renamed to “Dashboard,” confusing it with Progress Dashboard
Unsafe Large Patch	Too many changes at once	Broad prompts modified multiple files across features
Overengineering	Unnecessary complexity too early	Suggested advanced systems before core stability
Testing Assumption	Passing tests hides deeper problems	Smoke tests passed while hidden usability and behavior risks remained
Scope Failure	Large task produces incomplete/unrelated output	Asked for Phases 3-10 skeleton, but Codex returned only gamification instructions

6.1 Hallucinated Code and Context Errors

Hallucinated code and context errors occur when the AI assistant suggests code that does not match the actual project. This may include nonexistent functions, incorrect file names, or assumptions about project structure. These errors are dangerous because they can lead to wrong code placement, broken imports, or disconnected features.

6.2 Integration and Dependency Errors

Integration and dependency errors occur when one part of the project depends on another part that is missing, misconfigured, or incompatible. In the NeuroNova experiments, the Wikipedia integration issue showed that a local file existed, but its third-party dependency was missing from `requirements.txt`.

6.3 Product Drift and Decision-Making Limitations

Product drift occurs when the AI changes the direction of the requested task. This happened when the requested Smart Flashcard Scheduler / Spaced Repetition feature was replaced by a Flashcard Analytics Dashboard suggestion. This shows that AI tools can produce useful-looking output while failing to follow the actual requirement.

Decision-making limitation is related to this problem. AI tools can generate implementation quickly, but they cannot reliably decide product priorities, architecture direction, or long-term maintainability without human control.

6.4 Hidden Logic and Behavior Risks

Some risks are not detected by compilation or smoke tests. For example, the Study Planner integration passed tests, but controlled review found that `normalized[:7]` could silently remove manual or completed tasks. Similarly, the refactor into `neuronova_study_signals.py` passed tests, but controlled review found a subtle Exam Mode behavior drift.

6.5 Usability Risks

UI-focused AI changes may pass technical tests while still creating usability problems. The U6 UI redesign passed compilation and smoke tests, but controlled review found that renaming Home to “Dashboard” could confuse users because NeuroNova also had a separate Progress Dashboard page.

This taxonomy shows that AI-assisted coding risk is broader than syntax errors. In larger projects, the main risks include hidden behavior changes, unclear product decisions, dependency problems, and usability issues.

7. Results

The study recorded 16 AI-assisted development experiments in the NeuroNova project. The dataset includes 10 controlled experiments and 6 uncontrolled experiments.

A cleaned experiment-level summary is provided in Appendix A to improve transparency and allow readers to inspect how each experiment contributed to the final comparison.

7.1 Experiment Summary

Table 6. Controlled and uncontrolled experiment summary.

Type	Controlled	Uncontrolled	Total
Number of experiments	10	6	16
Successful	9	4	13
Failed or rejected	1	2	3

In the recorded experiments, the controlled workflow achieved a 90% observed success rate, while the uncontrolled workflow achieved a 66.7% observed success rate. Because the dataset is small and based on one project, these percentages should not be interpreted as broad statistical proof. Instead, they provide early case-study evidence that controlled workflows may be more predictable in this project context, while uncontrolled workflows can succeed but often create broader risk exposure.

7.2 Metrics Summary

Table 7. Metrics summary for controlled and uncontrolled workflows.

Metric	Controlled	Uncontrolled
Total recorded tasks	10	6
Successful tasks	9	4
Failed or rejected tasks	1	2
Success rate	90%	66.7%
Failure/rejection rate	10%	33.3%
High/Critical severity cases	1	2
Medium hidden-risk findings	3	0
Product drift cases	0	1
Scope failure cases	0	1
Hidden risks found by review	3	0
Minimal targeted fixes successful	4	0
Broad successful integrations/ refactors/redesigns	0	4

The controlled experiments were especially useful for read-only diagnosis, controlled review, and minimal targeted fixes. The uncontrolled experiments showed that broad prompts can succeed, but they often involve larger changes, multiple files, or core system areas.

7.3 Evidence Chains

The strongest evidence in the study comes from experiment chains where a broad or risky change was followed by controlled review and minimal correction.

Table 8. Evidence chains showing broad changes followed by controlled review.

Chain	Pattern	Meaning
C7 → C8 → C9	Problem revealed → controlled diagnosis → minimal dependency fix	Controlled debugging identified and fixed a dependency problem safely

Chain	Pattern	Meaning
U4 → C10 → C11	Broad Study Planner integration → controlled review → minimal preservation fix	Successful integration still contained hidden data-preservation risk
U5 → C12 → C13	Broad refactor → controlled review → compatibility fix	Refactor passed tests but caused subtle behavior drift
U6 → C14 → C15	Broad UI redesign → design review → label fix	UI changes passed tests but created usability ambiguity
U7	Broad provider/fallback refactor → review still needed	Core provider logic changes passed tests but require further controlled review
U8	Large multi-phase prompt → scope failure	Large uncontrolled prompts can collapse into incomplete or unrelated output

These chains show that AI-generated changes may appear successful at first but still require review for hidden risks.

7.4 Key Findings

The experiments produced five main findings.

Finding 1: Read-only controlled prompts are low risk.

C6 and C8 showed that asking the AI assistant to inspect without editing can produce useful analysis without changing or breaking the project.

Finding 2: Controlled minimal fixes are safer than broad follow-up changes.

C9, C11, C13, and C15 showed that small targeted fixes can solve specific problems without unnecessary refactoring.

Finding 3: Uncontrolled prompts can succeed but increase risk exposure.

U4, U5, U6, and U7 succeeded and passed tests, but they touched multiple files, created new shared logic, changed UI labels, or affected provider logic.

Finding 4: Passing tests is not enough.

C10, C12, and C14 found hidden risks after successful AI-generated changes. These risks were not syntax errors, so they would not necessarily be caught by compilation or smoke tests.

Finding 5: Large prompts can cause scope failure.

U8 showed that asking the AI assistant to handle too many phases across multiple files caused incomplete and unrelated output. This supports the need for smaller task decomposition.

Together, these findings support the main argument of this case study: AI coding assistants can be useful in student software projects, but their use should be bounded by a controlled workflow that separates inspection, implementation, review, and correction.

7.5 Summary of Findings by Research Question

RQ1 asked what failure modes appear when AI coding assistants are used in a larger student software project. The observed failure modes included hallucinated code, context errors, integration and dependency problems, product drift, decision-making limitations, data-preservation risk, behavior compatibility risk, UI label ambiguity, unsafe broad patches, testing assumptions, and scope failure.

RQ2 asked how controlled and uncontrolled AI-assisted workflows differ. In the recorded experiments, controlled workflows showed a higher observed success rate and were especially useful for diagnosis, hidden-risk detection, and minimal targeted fixes. Uncontrolled workflows sometimes succeeded, but they created broader risk exposure because they could affect multiple files, shared logic, user interface behavior, or core provider logic.

RQ3 asked what practical workflow can help student developers use AI coding assistants more safely. The findings support a controlled workflow based on six steps: Inspect → Diagnose → Implement minimally → Test → Review → Fix narrowly. This workflow keeps the human developer responsible for accepting, rejecting, or narrowing AI-generated changes.

7.6 Interpretation of Results

The results should be interpreted as evidence of workflow tradeoffs rather than as a causal statistical comparison. Controlled workflows were more successful in this case study partly because they limited the AI assistant's action space and made changes easier to review. Uncontrolled workflows sometimes succeeded, but they increased review difficulty because they touched broader system areas and created larger patches.

The most important result is therefore not only the difference between 90% and 66.7% observed success rates. The more important finding is that controlled workflows improved reviewability, reversibility, and hidden-risk detection. This suggests that workflow design may be a practical safety mechanism for student developers using AI coding assistants in larger projects.

8. Discussion

The results provide early case-study evidence that the main risk of AI-assisted development is not only incorrect code generation. In larger student software projects, the deeper problem is that AI coding assistants may lack reliable project-level judgment. They can generate implementation quickly, but they do not always understand architecture, product direction, user needs, dependency relationships, or long-term maintainability. This finding directly relates to RQ1, which asked what failure modes appear when AI coding assistants are used in a larger student software project.

8.1 Controlled Workflows Reduce Risk Exposure

The controlled workflow reduced risk by limiting what the AI assistant was allowed to do. Read-only inspection tasks such as C6 and C8 were especially safe because they allowed the AI to analyze the project without modifying files. Minimal fixes such as C9, C11, C13, and C15 were also safer because they changed only the specific part of the project needed to solve a diagnosed problem.

This suggests that AI coding assistants are most useful when their role is narrow and clearly defined. Instead of asking the AI to improve a whole system, student developers should ask it to inspect, diagnose, explain, or apply one small targeted change.

This finding relates to RQ2 because it explains why controlled workflows may have produced more predictable outcomes in the recorded experiments. The controlled workflow limited the AI assistant's action space, which made changes easier to inspect, test, and reverse.

8.2 Uncontrolled Prompts Can Succeed but Remain Risky

The uncontrolled experiments show that broad prompts do not always fail. U4, U5, U6, and U7 succeeded and passed tests. This is important because the argument of this paper is not that uncontrolled prompts always produce bad results.

The issue is risk exposure. Successful uncontrolled prompts often touched multiple files, created new shared logic, changed user interface behavior, or affected core AI provider logic. These changes may work technically but still require review because they can introduce hidden behavior changes, usability problems, or architecture risks.

Therefore, workflow safety should not be judged only by whether the code runs. It should also consider patch size, number of files changed, affected system area, reversibility, and whether the human developer understands the change.

This finding also relates to RQ2. The uncontrolled workflow did not always fail, but it exposed the project to larger changes and more difficult review conditions. In this case study, the difference between controlled and uncontrolled workflows was therefore not only success rate, but also risk visibility and review difficulty.

8.3 Passing Tests Is Not Enough

Several experiments showed that compilation and smoke tests are useful but incomplete. U4 passed tests, but C10 later found a data-preservation risk. U5 passed tests, but C12 found behavior compatibility drift. U6 passed tests, but C14 found UI label ambiguity.

These examples show that tests can confirm that the application still runs, but they may not detect hidden risks related to user data, expected behavior, navigation clarity, or product alignment. Controlled review is therefore necessary after broad AI-generated changes.

8.4 Human Decision-Making Remains Central

The findings support the idea that AI coding assistants should not control project direction. AI tools can help generate code, explain errors, and suggest improvements, but the human developer must decide what should be built, when it should be built, and whether the change fits the project.

This is especially important for student developers. Students may be tempted to accept AI-generated code because it looks professional or passes basic tests. However, without careful review, AI-generated changes can increase complexity, create hidden risks, or move the project away from its original purpose.

8.5 Implications for Student Developers

The NeuroNova case study suggests several practical lessons for students using AI coding assistants. Students should use Git checkpoints before AI-assisted changes, ask for read-only inspection before implementation, break large tasks into smaller controlled tasks, avoid broad prompts such as “fix everything” or “improve the whole project,” require the AI to identify the exact file and function before editing, prefer minimal patches over large replacements, test the application after every change, review successful AI-generated code for hidden risks, and keep product vision and user experience under human control.

These practices can help students benefit from AI assistance while reducing the chance of damaging larger projects.

These implications answer RQ3 by translating the observed failure modes and workflow differences into a practical mitigation strategy. The proposed workflow does not require advanced infrastructure, so it can be used by student developers who are still learning software engineering practices.

8.6 Implications for Research

For researchers, the case study suggests that AI-assisted development should be evaluated not only by task completion or productivity, but also by workflow control, risk exposure, reviewability, and hidden-risk detection. Future empirical studies should compare AI-assisted workflows using richer evaluation criteria, including patch size, number of affected files, reversibility, developer understanding, and post-generation review outcomes.

The study also suggests that student software projects are useful empirical settings for studying human-AI collaboration because they combine authentic development work with visible learning, decision-making, and workflow-control challenges.

9. Proposed Controlled Workflow

Based on the NeuroNova case study, this paper proposes a controlled AI-assisted development workflow for student software projects. The purpose of the workflow is to help students benefit from AI coding assistants while reducing risks related to hallucinated code, unsafe patches, dependency errors, behavior drift, and product misalignment.

This workflow aligns with the broader need for human oversight in human-AI collaboration during software engineering [10], [11].

The workflow is:

Inspect → Diagnose → Implement minimally → Test → Review → Fix narrowly

9.1 Step 1: Inspect

Before allowing the AI assistant to modify code, the developer should ask it to inspect the current project state. The AI should identify the relevant files, functions, dependencies, and possible causes of the issue without making changes.

Example prompt:

Inspect the current error. Do not edit files. Identify the exact file, function, and likely cause.

This step reduces context errors and prevents the AI from suggesting code for nonexistent files or functions.

9.2 Step 2: Diagnose

After inspection, the AI should explain the problem and identify the smallest safe solution. The diagnosis should separate symptoms from root causes.

Example prompt:

Explain the root cause of this issue and recommend the smallest safe fix. Do not edit files yet.

This step helps the student understand the problem before accepting any code change.

9.3 Step 3: Implement Minimally

When implementation is needed, the AI should make the smallest possible change. The prompt should limit the file scope and prevent unrelated refactoring.

Example prompt:

Fix only this issue in this file. Do not refactor. Do not rename functions. Preserve existing behavior.

Minimal implementation reduces patch size and makes review easier.

9.4 Step 4: Test

After implementation, the developer should run relevant tests. These may include Python compilation checks, root file compilation, smoke tests, manual feature testing, and dependency checks.

Example tests:

```
python -m py_compile file_name.py
python -m streamlit run main.py
```

Testing should not only check whether the application starts. It should also check whether the changed feature still behaves as expected and whether related features were affected.

9.5 Step 5: Review

Even if the code passes compilation and smoke tests, the developer should review the change before accepting it. The review should check for hidden risks such as data loss, behavior drift, dependency problems, confusing user interface changes, product misalignment, and unnecessary complexity.

Example review prompt:

Review this change for hidden risks. Check whether it preserves existing behavior, avoids data loss, avoids unnecessary refactoring, and fits the project goal. Do not edit files.

This step is important because several NeuroNova experiments showed that AI-generated changes could pass tests while still creating hidden risks.

9.6 Step 6: Fix Narrowly

If review finds a problem, the follow-up fix should correct only the identified issue. The developer should avoid starting another broad AI-generated change because broad follow-up prompts can create new risks.

Example fix prompt:

Fix only the identified issue. Do not refactor. Do not change unrelated files. Preserve the existing behavior.

This final step keeps the correction small, reviewable, and reversible.

10. Threats to Validity

This study has several threats to validity. These threats are important because the study is based on a small empirical case study rather than a large controlled experiment.

10.1 Construct Validity

Construct validity concerns whether the study measures the intended concepts accurately. In this research, concepts such as success, failure, rejection, hidden risk, product drift, and behavior compatibility risk involve human judgment. Although the paper defines these categories, some classifications may still be subjective. To reduce this risk, each experiment was recorded using a structured log that included the AI tool, workflow type, task summary, files changed, failure mode, severity, result, acceptance status, and notes. Future work should use multiple independent reviewers and inter-rater agreement to improve classification reliability.

10.2 Internal Validity

Internal validity concerns whether the observed differences between controlled and uncontrolled workflows can be attributed to the workflow itself. The success-rate comparison should not be interpreted as causal proof that controlled workflows always outperform uncontrolled workflows. The controlled and uncontrolled tasks differed in scope, difficulty, and purpose. Controlled tasks were often narrower and review-oriented, while uncontrolled tasks often involved broader integration, refactoring, or redesign. Therefore, the results are best interpreted as evidence of risk patterns and workflow tradeoffs rather than as a controlled statistical comparison.

10.3 External Validity

External validity concerns whether the findings generalize beyond this case study. The dataset is based on 16 recorded experiments from one student-built project, NeuroNova. Other projects may use different programming languages, architectures, AI tools, team structures, testing practices, and development goals. The findings may therefore not generalize directly to all student projects or professional software teams. However, the observed failure modes may still be useful for student developers working on larger evolving projects with AI coding assistants.

10.4 Reliability

Reliability concerns whether another researcher could repeat the study and reach similar conclusions. Because the experiments were collected during real development work, the study reflects realistic AI-assisted development conditions, but it is not perfectly controlled. Future work should improve reliability by publishing anonymized prompts, experiment logs, classification rubrics, accepted/rejected outputs, and test results. A stronger replication study could apply the same controlled workflow across multiple student projects and compare results using the same evaluation criteria.

10.5 Testing Limitations

Compilation and smoke tests were used when available, but the testing process was limited. These tests helped identify syntax errors and startup failures, but they could not fully detect data loss, usability ambiguity, behavior drift, product misalignment, or long-term maintainability problems. Future work should include unit tests, integration tests, regression tests, and user testing to evaluate AI-generated changes more rigorously.

Despite these threats to validity, the study provides useful early evidence from a real student software project. It shows that AI-assisted development risk is not limited to syntax errors and that larger student projects require workflow control, review, testing, and human decision-making.

11. Conclusion

This paper presented an empirical case study of failure modes and mitigation strategies for AI-assisted development in a larger student software project. Using NeuroNova as the case study, the research analyzed 16 recorded AI-assisted development experiments and compared controlled AI-assisted development with uncontrolled AI-assisted coding.

The results suggest that AI coding assistants can be useful for debugging, refactoring, feature integration, UI improvement, and code review. However, they also create risks when used without enough workflow control. The observed risks included hallucinated code, context errors, dependency problems, product drift, hidden logic risks, behavior compatibility issues, usability ambiguity, unsafe broad changes, and scope failure.

Within the recorded experiments, the controlled workflow achieved a higher observed success rate than the uncontrolled workflow. However, because the dataset is small and based on one project, this comparison should be interpreted as early case-study evidence rather than broad statistical proof. More importantly, controlled workflows helped identify hidden risks and apply minimal targeted fixes. The results also suggest that uncontrolled prompts can succeed, but they may create higher risk exposure when they touch multiple files, affect architecture, or make product-level decisions.

The main contribution of this paper is a practical controlled AI-assisted development workflow for student developers:

Inspect → Diagnose → Implement minimally → Test → Review → Fix narrowly

This workflow does not remove the need for human judgment. Instead, it helps students use AI coding assistants more safely by keeping the human developer responsible for architecture, testing, product direction, and final decisions.

Future work should test the workflow on more student projects, involve more AI coding tools, use independent reviewers for failure classification, and include stronger automated testing. Future research could also study whether controlled AI-assisted development improves student learning, debugging ability, and software engineering discipline over time.

Overall, the study suggests that student developers should treat AI coding assistants as controlled coding partners rather than autonomous software engineers. While the evidence is limited to one project, the observed patterns show why workflow discipline, review, testing, and human decision-making remain essential when AI-generated changes are introduced into larger student software systems.

Appendix A: Experiment Log Summary

This appendix summarizes the 16 recorded AI-assisted development experiments used in the study. The table improves transparency by showing how each experiment contributed to the controlled and uncontrolled workflow comparison.

The full experiment-level details are provided in Appendix A so that the main paper can remain focused on the core analysis while preserving transparency.

Table A1. Cleaned experiment-level summary used in the case study.

C6 - Controlled - Config / Debugging

- **Result:** Successful
- **Failure mode / risk:** None
- **Severity:** None
- **Accepted:** Yes
- **Summary:** Codex followed a read-only controlled prompt and recommended a low-risk `config.py` sanity check without editing files.

C7 - Controlled - Feature Addition

- **Result:** Failed / rejected
- **Failure mode / risk:** Integration Error / Dependency Error
- **Severity:** Critical
- **Accepted:** No
- **Summary:** Codex provided manual paste instructions for a gamification leaderboard, but running the app revealed a `ModuleNotFoundError` related to `wikipedia_api`.

C8 - Controlled - Bug Diagnosis

- **Result:** Successful review
- **Failure mode / risk:** Dependency issue identified
- **Severity:** None
- **Accepted:** Yes
- **Summary:** Codex inspected the missing `wikipedia_api` import error without editing files and identified that the third-party `wikipedia` package was missing from `requirements.txt`.

C9 - Controlled - Minimal Bug Fix

- **Result:** Successful
- **Failure mode / risk:** None
- **Severity:** None
- **Accepted:** Yes
- **Summary:** Codex applied the smallest safe fix by adding wikipedia to `requirements.txt`; dependency import testing and `py_compile` passed.

U3 - Uncontrolled - Feature Addition

- **Result:** Failed / rejected
- **Failure mode / risk:** Context Error / Hallucinated Code / Product Drift / Decision-Making Limitation
- **Severity:** High
- **Accepted:** No
- **Summary:** The AI was asked to add spaced repetition but shifted toward a Flashcard Analytics Dashboard and suggested unverified functions and project structure.

U4 - Uncontrolled - Feature Integration

- **Result:** Successful
- **Failure mode / risk:** Broad integration risk
- **Severity:** None
- **Accepted:** Yes
- **Summary:** Codex broadly integrated Study Planner with flashcards, Exam Mode, and the progress dashboard; compilation and smoke tests passed, but later controlled review found hidden data-preservation risk.

C10 - Controlled - Code Review

- **Result:** Successful review
- **Failure mode / risk:** Data Preservation Risk / Hidden Logic Risk
- **Severity:** Medium
- **Accepted:** Yes
- **Summary:** Codex reviewed the U4 integration without editing files and found that `normalized[:7]` could silently drop manual or completed planner tasks.

C11 - Controlled - Minimal Bug Fix

- **Result:** Successful
- **Failure mode / risk:** None
- **Severity:** None

- **Accepted:** Yes
- **Summary:** Codex edited only `neuronova_study_planner.py` to preserve manual and completed tasks while limiting appended smart tasks; tests passed.

U5 - Uncontrolled - Refactoring

- **Result:** Successful
- **Failure mode / risk:** Broad refactor risk
- **Severity:** None
- **Accepted:** Yes
- **Summary:** Codex performed a broad refactor by creating `neuronova_study_signals.py` and modifying multiple modules; tests passed, but later review found behavior drift.

C12 - Controlled - Code Review

- **Result:** Successful review
- **Failure mode / risk:** Behavior Compatibility Risk / Hidden Logic Risk
- **Severity:** Medium
- **Accepted:** Yes
- **Summary:** Codex reviewed the U5 refactor and found that Exam Mode fallback behavior had changed from the previous `file_type` behavior.

C13 - Controlled - Minimal Bug Fix

- **Result:** Successful
- **Failure mode / risk:** None
- **Severity:** None
- **Accepted:** Yes
- **Summary:** Codex edited only `neuronova_exam_mode.py` to preserve the old Exam Mode fallback behavior; compilation and smoke tests passed.

U6 - Uncontrolled - UI Redesign

- **Result:** Successful
- **Failure mode / risk:** Broad UI redesign risk
- **Severity:** None
- **Accepted:** Yes
- **Summary:** Codex redesigned several UI-related files; tests passed, but later controlled design review found navigation label ambiguity.

C14 - Controlled - Design Review

- **Result:** Successful review

- **Failure mode / risk:** UI Label Ambiguity / Usability Risk
- **Severity:** Medium
- **Accepted:** Yes
- **Summary:** Codex reviewed the U6 redesign without editing files and found that renaming Home to Dashboard could confuse users because Progress was also dashboard-like.

C15 - Controlled - UI Fix

- **Result:** Successful
- **Failure mode / risk:** None
- **Severity:** None
- **Accepted:** Yes
- **Summary:** Codex edited only `neuronova_ui.py` and changed the visible sidebar label from Dashboard back to Home without changing routing or feature logic.

U7 - Uncontrolled - Provider Refactor

- **Result:** Successful
- **Failure mode / risk:** Broad core-system change risk
- **Severity:** None
- **Accepted:** Yes
- **Summary:** Codex broadly improved the AI provider and fallback system by changing provider-management files; tests passed, but the change affected core logic and required review.

U8 - Uncontrolled - Large Multi-Phase Skeleton

- **Result:** Failed / rejected
- **Failure mode / risk:** Task Mismatch / Context Loss / Incomplete Output / Scope Failure
- **Severity:** High
- **Accepted:** No
- **Summary:** Codex failed to complete the requested Phases 3-10 skeleton and returned mostly gamification leaderboard instructions; the output was rejected.

This appendix reflects the final cleaned dataset of 10 controlled experiments and 6 uncontrolled experiments.

References

- [1] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” arXiv preprint arXiv:2302.06590, 2023.

- [2] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv preprint arXiv:2107.03374, 2021.
- [3] P. Runeson and M. Höst, “Guidelines for Conducting and Reporting Case Study Research in Software Engineering,” *Empirical Software Engineering*, vol. 14, pp. 131–164, 2009.
- [4] OpenAI, “Introducing ChatGPT,” OpenAI, 2022.
- [5] GitHub, “Research: Quantifying GitHub Copilot’s impact on developer productivity and happiness,” GitHub Blog, 2022.
- [6] K. Z. Cui, M. Demirer, S. Jaffe, L. Musolff, S. Peng, and T. Salz, “The Effects of Generative AI on High Skilled Work: Evidence from Three Field Experiments with Software Developers,” 2024.
- [7] C. Bull and J. Kharrufa, “Generative AI Assistants in Software Development Education,” arXiv preprint arXiv:2303.13936, 2023.
- [8] C. Sengul, “Software engineering education in the era of conversational AI,” 2024.
- [9] W. Takerngsaksiri et al., “Students’ Perspectives on AI Code Completion,” arXiv preprint arXiv:2311.00177, 2023.
- [10] M. Hamza et al., “Human-AI Collaboration in Software Engineering,” arXiv preprint arXiv:2312.10620, 2023.
- [11] C. Treude et al., “How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering,” 2025.
- [12] J. Ji et al., “Cybersecurity Risks of AI-Generated Code,” Center for Security and Emerging Technology, 2024.
- [13] Y. Fu et al., “Security Weaknesses of Copilot-Generated Code in GitHub,” 2025.

AI Use Disclosure

AI tools, including ChatGPT and Codex, were used during the development and research process for coding assistance, debugging suggestions, drafting support, proofreading, workflow comparison, and research organization.

All AI-assisted outputs were reviewed by the author. The experiment log records the AI tool used, workflow type, task description, result, failure mode, severity, and notes.

The author remained responsible for selecting the case study, validating results, interpreting findings, organizing the research, and preparing the final submission.